

For personal use only,
please do not
distribute on
non-UBC domains.

04b - Test Robustness

CPSC310

Nick Bradley & Kyle Chin



Learning Goals

By the end of the lecture you should be able to:

- Design *stubs* and *spies* that stand in for real dependencies, to *control* what the code under test receives and *observe* what it sends.
- Apply the *Liskov substitution principle (LSP)* to decide whether a test double is a valid substitute.
- Systematically create comprehensive test suites using *Equivalence Class Partitioning (ECP)*
- Evaluate the strength of a test suite using coverage

Recap: Testing Challenges...

Tests are particularly hard to write when the CUT is ...

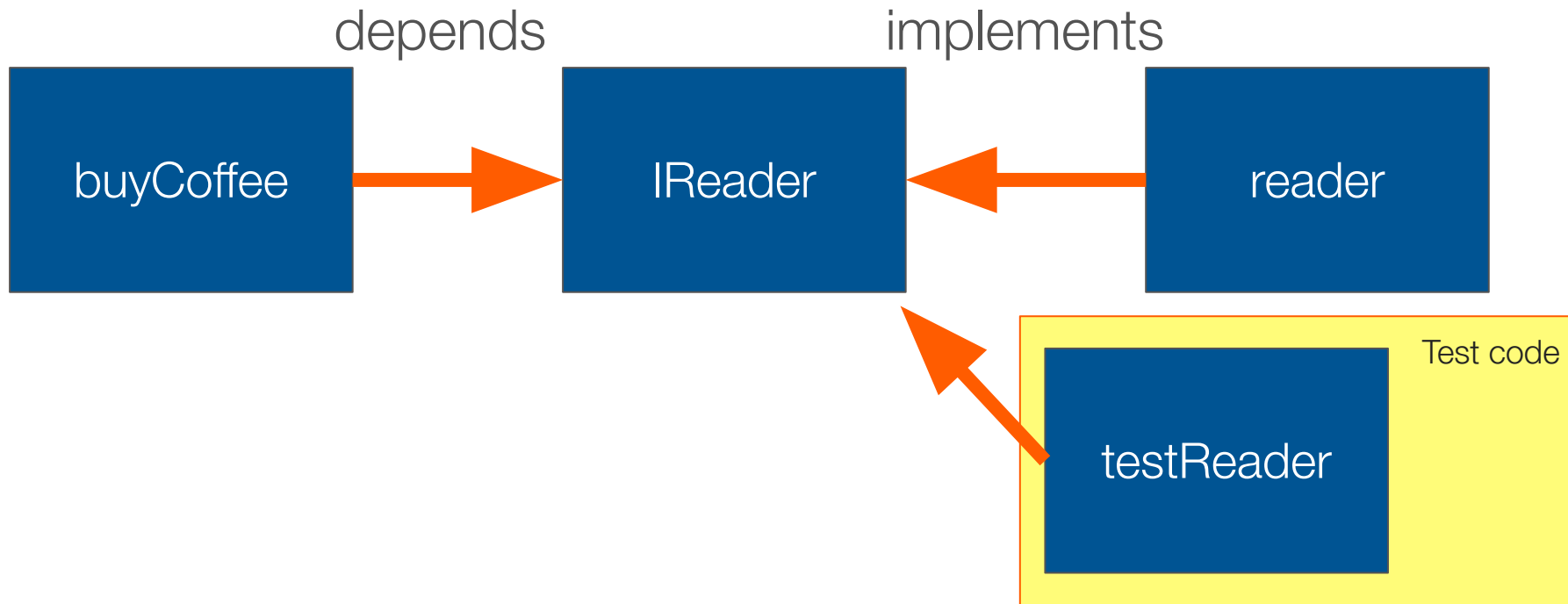
- ***uncontrollable***: a bank, a clock
- ***unobservable***: you can't see what was sent
- ***unsafe or slow***: real money, the network

Test Doubles

- A test double stands in for a dependency that the code under test uses.
- A *stub* controls what the dependency gives back, like our approving and declining readers.
- A *spy* records what the code under test sent to it, like our recording reader.

DIP enables test doubles

“Depend upon *abstractions*, do not depend upon *implementations*.”



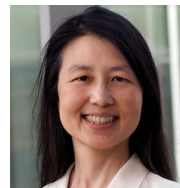
Liskov Substitution Principle

Subtype Requirement: Let $\phi(x)$ be a property provable about objects x of type T . Then $\phi(y)$ should be true for objects y of type S where S is a subtype of T .

[Barbara Liskov and Jeanette Wing, A Behavioral Notion of Subtyping, ACM TOPLAS, 18:11-18:41, 1994.]



Barbara Liskov



Jeanette Wing

What is LSP?

Substitutability:

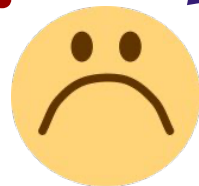
A subclass should not break the expectations set by its superclass!

```
class GoodPerson {  
    public freeBird(b: Bird) {  
        b.flyFromHeight(100);  
        // ... watch bird fly!  
    }  
}
```

```
class Bird {  
    // all heights work!  
    public flyFromHeight(height: number) {  
        // ... some implementation  
    }  
}
```



```
class Seagull extends Bird {  
    // all heights work!  
    public flyFromHeight(height: number) {  
        // ... some implementation  
    }  
}
```



```
class Penguin extends Bird {  
    // heights under 50cm only!  
    public flyFromHeight(height: number) {  
        // ... another implementation  
    }  
}
```



LSP Methods Rule

Substitutability:
A subclass
should not break
the expectations
set by its
superclass!

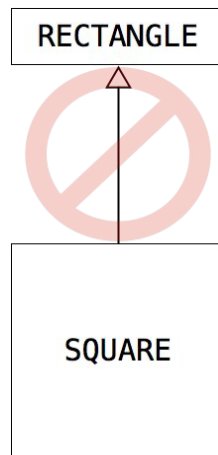
Precondition Rule

Preconditions should
not be strengthened
(for the same inputs)
OK to widen!

Postcondition Rule

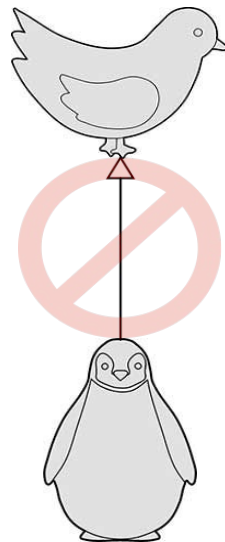
Postconditions should
not be weakened (for
the same inputs)
OK to narrow!

Resize only lets
you specify one
dimension.



Changing width will
also change height.
Bad: widened
postcondition!

Can fly.



Additional effect:
Penguin perishes
Bad: stronger
precondition!

A young girl with red hair in pigtails, wearing a red dress, looking upwards with a concerned expression against a bright, hazy orange and yellow background.

Did you **create a test double?**

A close-up of Thanos, a purple-skinned character with a long white beard, looking slightly to the side with a serious expression against a hazy orange and yellow background.

Yes.

A young girl with red hair in pigtails, wearing a red dress, looking upwards with a concerned expression against a bright, hazy orange and yellow background.

What did it cost?

A close-up of Thanos, a purple-skinned character with a long white beard, looking slightly to the side with a serious expression against a hazy orange and yellow background.

Everything

Recap: Testability with test doubles

- To test code that calls something hard to test, depend on an interface and pass the dependency in (DIP).
- Test doubles stand in for it. *Stubs* control what comes back, and *spies* record what goes out.
- A double has to keep the interface's contract, not just its shape (LSP).
- **The cost:** another layer; doubles can drift from the real thing & need to be updated when the interface changes. The real implementation still needs tests of its own.

What is a
strong test
suite?

What makes a test suite strong?

It fails when the implementation is wrong. A strong test suite has:

The right inputs: every behaviour in the spec has a test

→ Equivalence classes + boundary values

Precise checks: each test would notice a wrong answer

→ Assertions that actually verify what the spec guarantees

Reliable results: the same results every run

→ Deterministic code or test doubles

Coverage of (most) behaviour in the code

Building a strong test suite

Spec tests <<< ONLY
TEST what is in the spec.

Get the
requirements.

Turn into stubs
with
specifications.

Write behavioural
tests to check
specifications.

Implement until
all behavioural
tests are passing.

Check how good
your test suite is for
this specific
implementation.
Check coverage,
(Check mutants)

Write structural
tests to
exercise the full
codebase.

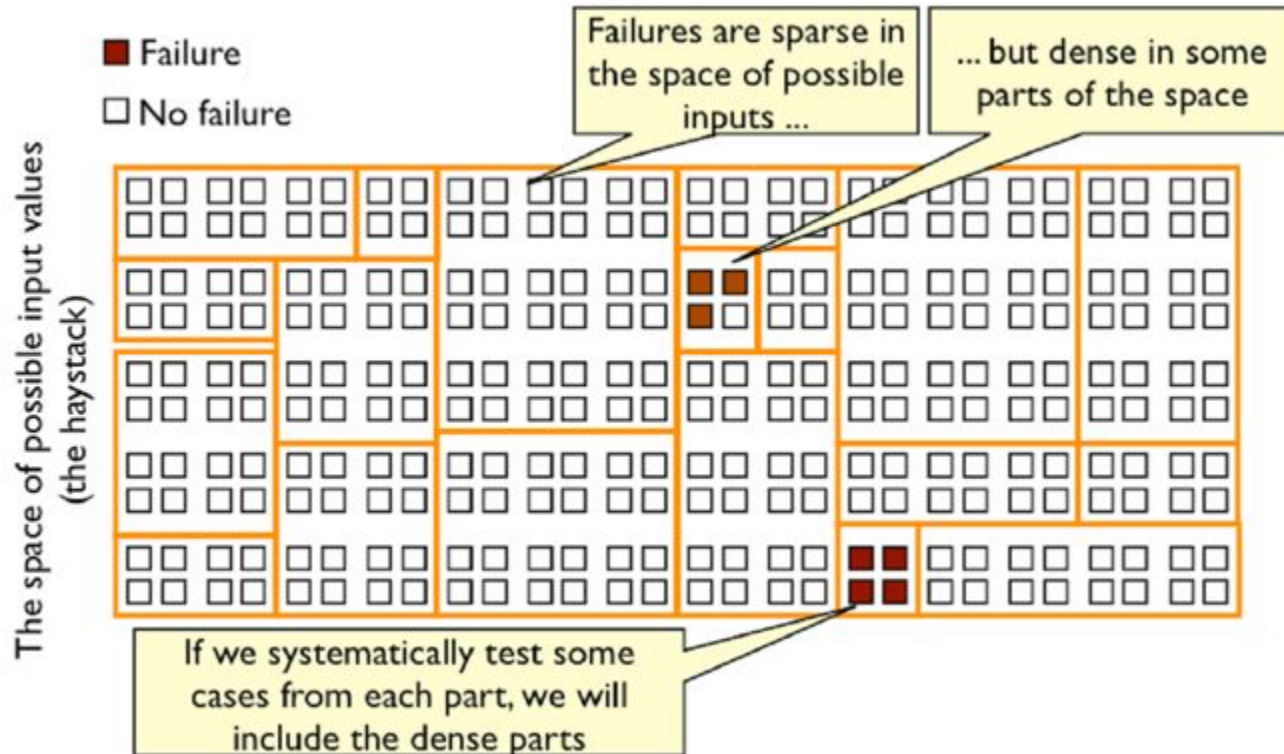
Equivalence Class Partitioning (ECP)

`getLetterGrade()` accepts any number. We can't test them all, so which do we pick?

- Split inputs into classes that treat the spec the same way
- Test one value per class: if 72 gets “B”, 71 probably does too
- Work from the spec, not the code: black-box technique

ECP finds the cases you missed, and shows which tests are repeats

Equivalence class partitioning



Identifying equivalence classes

An equivalence class is a set of values the spec treats the same way.

Input domain:

- Starts from the parameter types in the function signature
- Narrowed by preconditions: excluded values aren't tested
- Split into *valid* & *invalid* classes: invalid = rejected by the spec

Output range:

- Starts from the return type, plus any documented errors
- Split by the postconditions: what each input should produce
- Split into *normal* & *error* classes: errors are promised outputs

Partition the input space

```
/**
 * Converts a percentage grade to a letter grade.
 *
 * | Letter | Percentage |
 * | ----- | ----- |
 * | A      | 80-100    |
 * | B      | 68-79     |
 * | C      | 55-67     |
 * | D      | 50-54     |
 * | F      | 0-49      |
 *
 * @param grade Grade to convert, in percent. Valid grade 0-100.
 * @returns The letter grade: "A", "B", "C", "D", or "F".
 * @throws RangeError if grade is below 0 or above 100.
 */
getLetterGrade(grade: number): string;
```

Partition the input space

```
/**  
 * Converts a percentage grade to a letter grade.  
 *  
 * | Letter | Percentage |  
 * | ----- | ----- |  
 * | A      | 80-100    |  
 * | B      | 68-79     |  
 * | C      | 55-67     |  
 * | D      | 50-54     |  
 * | F      | 0-49      |  
 *  
 * @param grade Grade to convert, in percent. Valid grade 0-100.  
 * @returns The letter grade: "A", "B", "C", "D", or "F".  
 * @throws RangeError if grade is below 0 or above 100.  
 */  
getLetterGrade (grade: number): string;
```

Valid input classes: $0 \leq \text{grade} \leq 100$

Partition the input space

```
/**
 * Converts a percentage grade to a letter grade.
 *
 * | Letter | Percentage |
 * | ----- | ----- |
 * | A      | 80-100    |
 * | B      | 68-79     |
 * | C      | 55-67     |
 * | D      | 50-54     |
 * | F      | 0-49      |
 *
 * @param grade Grade to convert, in percent. Valid grade 0-100.
 * @returns The letter grade: "A", "B", "C", "D", or "F".
 * @throws RangeError if grade is below 0 or above 100.
 */
```

Valid input classes: $0 \leq \text{grade} \leq 100$

Invalid input classes: $\text{grade} < 0$ & $\text{grade} > 100$

```
getLetterGrade(grade: number): string;
```

Partition the input space

```
/**
 * Converts a percentage grade to a letter grade.
 *
 * | Letter | Percentage |
 * | ----- | ----- |
 * | A      | 80-100    |
 * | B      | 68-79     |
 * | C      | 55-67     |
 * | D      | 50-54     |
 * | F      | 0-49      |
 *
 * @param grade Grade to convert, in percent. Valid grade 0-100.
 * @returns The letter grade: "A", "B", "C", "D", or "F".
 * @throws RangeError if grade is below 0 or above 100.
 */
getLetterGrade(grade: number): string;
```

Valid input classes: $0 \leq \text{grade} \leq 100$

Invalid input classes: $\text{grade} < 0$ & $\text{grade} > 100$

Not tested: "80", undefined, null, [], ...

Partition the output space

```
/**
 * Converts a percentage grade to a letter grade.
 *
 * | Letter | Percentage |
 * | ----- | ----- |
 * | A      | 80-100    |
 * | B      | 68-79     |
 * | C      | 55-67     |
 * | D      | 50-54     |
 * | F      | 0-49      |
 *
 * @param grade Grade to convert, in percent. Valid grade 0-100.
 * @returns The letter grade: "A", "B", "C", "D", or "F".
 * @throws RangeError if grade is below 0 or above 100.
 */
getLetterGrade(grade: number): string;
```



Letter	Percentage
A	80-100
B	68-79
C	55-67
D	50-54
F	0-49

Normal output classes: "A", "B", "C", "D", "F"

Partition the output space

```
/**
 * Converts a percentage grade to a letter grade.
 *
 * | Letter | Percentage |
 * | ----- | ----- |
 * | A      | 80-100    |
 * | B      | 68-79     |
 * | C      | 55-67     |
 * | D      | 50-54     |
 * | F      | 0-49      |
 *
 * @param grade Grade to convert, in percent. Valid grade 0-100.
 * @returns The letter grade: "A", "B", "C", "D", or "F".
 * @throws RangeError if grade is below 0 or above 100.
 */
getLetterGrade(grade: number): string;
```

Normal output classes: "A", "B", "C", "D", "F"

Error output classes: RangeError

Come up with test cases from partitions

Choose values so every class from both views has at least one.
A single value can cover an input partition and an output partition at the same time: 72 covers "valid" and "B".

Valid input classes: $0 \leq \text{grade} \leq 100$

Normal output classes: "A", "B", "C", "D", "F"

Invalid input classes: $\text{grade} < 0$ & $\text{grade} > 100$

Error output classes: `RangeError`



9 classes total

Come up with test cases from partitions

Choose values so every class from both views has at least one.
A single value can cover an input partition and an output partition at the same time: 72 covers "valid" and "B".

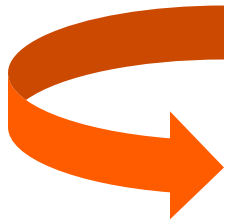
Valid input classes: $0 \leq \text{grade} \leq 100$

Normal output classes: "A", "B", "C", "D", "F"

Invalid input classes: $\text{grade} < 0$ & $\text{grade} > 100$

Error output classes: `RangeError`

9 classes total



Pick one value for grade for each class e.g.

-10, 25, 52, 60, 72, 90, 110

Add test cases at the boundary

Bugs cluster at the edges of partitions: $\geq$ instead of $>$, or A starting at 81 instead of 80. Test the value on each side of every edge.

Edge	Last value on one side	First value on the other
invalid/F	-1 $\rightarrow$ RangeError	0 $\rightarrow$ "F"
F/D	49 $\rightarrow$ "F"	50 $\rightarrow$ "D"
D/C	54 $\rightarrow$ "D"	55 $\rightarrow$ "C"
C/B	67 $\rightarrow$ "C"	68 $\rightarrow$ "B"
B/A	79 $\rightarrow$ "B"	80 $\rightarrow$ "A"
A/invalid	100 $\rightarrow$ "A"	101 $\rightarrow$ RangeError

Why partition both input and outputs?

For `getLetterGrade` they agree: each letter comes from one range of inputs. They diverge when an output depends on a combination of inputs.

Input Partitioning

Finds values the code might mishandle: invalid, empty, extreme

- `computeGPA`: a 1-credit and a 4-credit course both give 3.0

Output Partitioning

Finds behaviours no single input points to

- `computeGPA`: “can the grade go over 100?”
Leads to `points > maxPoints`, the case that throws

Coverage: A Quick Primer

We use coverage to answer:

How much of the code did our tests execute?

Key metrics:

- **Line coverage:** hit each executable line?
- **Statement coverage:** ran each statement?
- **Branch coverage:** took each decision outcome?

It's an essential tool for discovering *untested structural behaviour*.

Coverage tools

covered

not
covered

```
47 CALL FUNCTION 'SALC_CACHE_GET_MTE_BY_CLASS'
48   EXPORTING
49     request      = request
50     bypass_cache = bypass_cache
51   IMPORTING
52     result       = result
53   EXCEPTIONS
54     OTHERS       = 99.
55 IF sy-subrc <> 0.
56   RAISE salc_internal_error.
57 ENDIF.
58
59
60 READ TABLE result INDEX 1 ASSIGNING <fs_result>.
61
62 IF <fs_result>-rc >= al_rc_first_error.
63   CASE <fs_result>-rc.
64     WHEN al_rc_system_invalid. RAISE system_invalid.
65     WHEN al_rc_no_route.       RAISE system_not_available.
66     WHEN al_rc_internal_error. RAISE salc_internal_error.
67     WHEN al_rc_call_invalid.   RAISE salc_internal_error.
68     WHEN al_rc_group_not_in_repository.
69       RAISE group_not_found_in_repository.
70     WHEN al_rc_group_has_no_members. RAISE group_has_no_members.
71     WHEN OTHERS.                     RAISE other_problem.
72   ENDCASE.
73 ENDIF.
74
75 APPEND LINES OF <fs_result>-tidtbl TO tide_for_mteclass.
```

Using coverage tools effectively

Coverage measures **where your tests went**, not **what they validated**.

- does **not** check correctness
- can be fooled by tests without assertions
- does not consider the **input space**
- artificial thresholds/can be gamed

Use coverage to find what your tests missed. Then decide if more tests are needed.

Recap: test robustness

- Equivalence class partitioning helps us reason about what we should test to cover our specification
- Coverage helps us see what code we have executed in our test
- Use these to generate strong test suites